Git and GitHub Basics

1. Introduction

Git is a distributed version control system that tracks changes to files over time, allowing multiple people to collaborate on a project without overwriting one another's work. GitHub is a cloud-based platform that hosts Git repositories online and adds collaboration features on top of Git, such as issue tracking, pull requests, and code review. While Git is the underlying tool that runs on a local machine, GitHub is a service that stores and manages Git repositories on the web. Together, they form the backbone of modern software development workflows, used by individual students and large engineering teams alike.

2. Why Version Control Matters

Before version control systems existed, developers often kept multiple copies of a project (such as "project_final", "project_final2") to preserve earlier states of their work. This approach is error prone and does not scale when several people edit the same files. Version control solves this by:

- Recording a complete history of every change, who made it, and why.
- Allowing changes to be reverted safely if something breaks.
- Enabling multiple people to work on the same project in parallel through branches.
- Providing a single source of truth that can be synced across machines and collaborators.

3. Core Git Concepts

3.1 Repository

A repository, or "repo", is a directory that Git is tracking. It contains the project files along with a hidden .git folder that stores the entire history of changes. A repository can exist purely on a local machine or be linked to a remote copy hosted on a service like GitHub.

3.2 The Three Areas

Git organizes work through three conceptual areas: the working directory, where files are actively edited; the staging area (or index), where changes are prepared before being recorded; and the repository itself, where committed snapshots are permanently stored. Understanding this flow is key to using Git effectively.

3.3 Commits

A commit is a snapshot of the project at a specific point in time, accompanied by a message explaining the change. Commits form a chronological history that can be reviewed, compared, or reverted. Good practice is to make small, frequent commits with clear, descriptive messages rather than one large commit covering many unrelated changes.

3.4 Branches

A branch is an independent line of development. The default branch is traditionally called "main" . Creating a new branch allows a developer to experiment or build a feature without affecting the stable main branch. Once the work is complete and tested, the branch can be merged back.

4. Essential Git Commands

The table below and commands that follow cover the operations used in nearly every Git workflow.

```
git init                        # Initialize a new repository
git clone <url>                 # Copy a remote repository locally
git status                      # Show changed/staged files
git add <file>                  # Stage a file for commit
git commit -m "message"         # Save a snapshot with a message
git branch <name>               # Create a new branch
git checkout <branch>           # Switch to a branch
git merge <branch>              # Merge a branch into the current one
git pull                        # Fetch and merge remote changes
git push                        # Upload local commits to the remote
git log                         # View commit history
```

5. GitHub and Collaboration

GitHub extends Git with a web interface and collaboration tools. A remote repository hosted on GitHub acts as the shared point of truth that team members push to and pull from. Common GitHub-specific concepts include forking a repository to propose changes to a project one does not own, opening a pull request to request that changes be reviewed and merged, and using issues to track bugs, tasks, and discussions. Code review through pull requests allows teammates to comment on specific lines before changes are accepted, improving code quality and catching mistakes early.

6. A Typical Beginner Workflow

- Create a repository on GitHub, or clone an existing one to your machine.
- Create a new branch for the feature or fix you are working on.
- Make changes, then stage and commit them with clear messages.
- Push the branch to GitHub.
- Open a pull request describing the change.
- Address review feedback, then merge the pull request into main.

7. Glossary of Key Terms

Term	Meaning
Repository (repo)	A project's folder tracked by Git, containing all files and their full history.
Commit	A saved snapshot of changes, with a message describing what changed and why.
Branch	An independent line of development, allowing work without affecting the main codebase.
Clone	Downloading a full copy of a remote repository, including its history, to your machine.
Push	Uploading local commits to a remote repository such as GitHub.

Pull	Downloading and merging changes from a remote repository into your local branch.
Merge	Combining changes from one branch into another.
Pull Request (PR)	A GitHub feature to propose, review, and discuss changes before merging.
Fork	A personal copy of someone else's repository on GitHub, used to propose changes independently.
Clone vs Fork	Clone copies to your local machine; fork copies to your own GitHub account.